

PARTIE C : REFLEXION SYSTEME

1. Diagnostic : " J ai payé mais je n'ai pas accès au cours "

D'abord je regarde si la commande existe et son statut actuel (pending, paid, failed) dans la table **orders**. Si elle est déjà à paid, le problème n'est pas côté paiement mais côté livraison de l'accès, donc je vérifie les logs de la partie qui débloque le cours pour ce **order_id**.

Si la commande est encore à pending, je regarde dans **payment_events** si un webhook est arrivé pour ce **order_id**. Aucun événement du tout veut dire que l'agrégateur n'a jamais notifié notre serveur (souci réseau, mauvaise URL configurée côté agrégateur, ou webhook rejeté avant même d'arriver, par exemple un firewall).

Si un événement existe mais que la commande est restée à pending, je regarde le détail de cet événement : signature invalide (webhook rejeté par nous, potentiellement un souci de config du secret partagé), montant incohérent (flag **amount_mismatch**), ou statut **failed** envoyé par l'agrégateur. Ça permet de savoir si le blocage vient de notre validation ou d'un vrai échec de paiement côté agrégateur.

Enfin, je vérifie directement chez l'agrégateur (dashboard **Bictorys/Hub2**) que le paiement est bien passé de son côté. Si oui mais que rien n'est arrivé chez nous, c'est un souci d'intégration à corriger avec eux. Une fois la cause identifiée, je peux rejouer manuellement l'événement ou débloquer l'accès à la main en attendant le fix.

2. Performance : 3 leviers pour qu'une page Next.js charge vite en 3G

Je n'ai pas encore pratiqué Next.js en profondeur, je viens plutôt de **React** classique. En creusant la doc Next et en discutant avec Claude pour comprendre le fonctionnement du rendu serveur, voici les 3 leviers qui ressortent comme les plus impactants en 3G :

1. **Le rendu côté serveur (SSG ou ISR plutôt que CSR)** : la page arrive déjà construite en HTML, le téléphone n'a pas besoin de télécharger tout le JS puis exécuter le rendu avant d'afficher quoi que ce soit. En React classique tout se fait côté client, donc c'est le changement le plus important en venant de React.

2. **Les images** : le composant `next/image` redimensionne et sert des formats modernes (WebP/AVIF) automatiquement selon la taille d'écran, plutôt que de charger une image lourde pensée pour un écran desktop.

3. **Le JS chargé au départ** : ne charger que le JavaScript nécessaire à l'affichage initial (code splitting automatique par page dans Next, composants secondaires en chargement différé). Moins de JS à télécharger et à exécuter sur un téléphone d'entrée de gamme en 3G, c'est ce qui fait la plus grosse différence de ressenti.

3. Qualité : quelle stratégie de tests pour deux devs sans QA

Mon intuition de départ était que des tests fonctionnels (comme ceux qu'on a écrits sur le webhook et les inscriptions) plus quelques tests unitaires suffiraient. Après quelques lectures et vidéos sur le sujet, je pense que c'est globalement la bonne direction, mais avec une nuance importante sur où mettre l'effort.

Ce qu'on retrouve souvent, c'est l'idée de prioriser les tests là où une régression coûte cher : les chemins critiques comme le paiement, l'authentification, ou tout ce qui touche à l'argent ou aux données sensibles. Sur ces parties-là, des tests fonctionnels comme sur le webhook (idempotence, signature, montant) valent largement plus que des tests unitaires isolés sur chaque petite fonction.

Pour le reste (affichage, petites fonctions utilitaires), je pense qu'à 2 devs sans QA, viser une couverture à 100% partout est une perte de temps qui ralentit la vitesse de livraison sans vrai bénéfice. Le curseur réaliste : tests fonctionnels systématiques sur tout ce qui touche à l'argent et aux données utilisateur, tests unitaires ciblés sur la logique métier complexe (calculs, validations), et le reste couvert par une relecture de code croisée entre les deux devs plutôt que par des tests automatisés exhaustifs.

4. Dette technique : un raccourci acceptable et son signal de remboursement

Je n'ai jamais travaillé en startup, mais sur des projets réalisés en groupe pendant ma formation, on avait par exemple codé en dur les rôles utilisateurs (admin/membre) directement dans des `if/else` dans le code plutôt que de faire un vrai système de permissions en base de données. Sur le moment c'était le bon choix : on avait 2 rôles fixes, pas de besoin réel de flexibilité, et ça nous a fait gagner du temps pour livrer les fonctionnalités qui comptaient plus pour la démo.

Le signal qui indique qu'il faut rembourser cette dette : dès qu'on nous a demandé d'ajouter un 3ème rôle (modérateur), on s'est retrouvés à devoir modifier des conditions un peu partout dans le code plutôt que de juste ajouter une ligne dans une table de permissions. Concrètement, le signal c'est quand le raccourci commence à être touché à chaque nouvelle fonctionnalité au lieu d'être un cas isolé : à ce moment le coût de le garder devient plus élevé que le coût de le corriger proprement.